

Project Athena: Enterprise Resource Platform

Developer Integration & Architecture Guide

Bruce M.

Contents

1	System Overview	3
2	Backend Architecture (Java)	3
2.1	RESTful API Layer	3
2.2	Data Persistence Strategy	3
3	Frontend Architecture (Angular)	3
3.1	Dynamic Rendering	3
4	Production Integration Guide	3
4.1	1. Database Connection	4
4.2	2. Enabling JPA Repositories	4
5	Deployment Pipeline	4

1 System Overview

Project Athena is a decoupled, multi-tenant resource management platform. It is designed to replace legacy monolithic systems with a modern microservices architecture using **Java Spring Boot** (Backend) and **Angular** (Frontend).

This system operates in two modes:

- **Development Mode:** Uses in-memory mock data for rapid UI prototyping.
- **Production Mode:** Connects to a live PostgreSQL cluster for persistent storage.

2 Backend Architecture (Java)

The core business logic resides in the `/backend` directory, utilizing the Spring ecosystem.

2.1 RESTful API Layer

The `ResourceController` exposes strict JSON endpoints for external consumption.

```
1 // Example Endpoint: Fetch all resources
2 @GetMapping
3 public List<Resource> getAllResources() {
4     return resourceService.findAll();
5 }
```

2.2 Data Persistence Strategy

The application uses **Spring Data JPA** to abstract SQL interactions. The `Resource` entity maps directly to the `resources` table in the database, utilizing a `tenantId` column to ensure multi-tenant data isolation.

3 Frontend Architecture (Angular)

The user interface is a Single Page Application (SPA) located in `/frontend`.

3.1 Dynamic Rendering

Angular's `*ngFor` directive is used to iterate over JSON payloads returned by the API, rendering the `ResourceListComponent` table dynamically. State changes (e.g., Active vs. Migrating) are handled via CSS binding.

4 Production Integration Guide

CRITICAL: To transition this application from "Simulation" to "Live Production," the following configurations must be applied.

4.1 1. Database Connection

Locate `src/main/resources/application.properties` and update the datasource configuration to point to your live PostgreSQL instance:

```
1 # Production Database Config
2 spring.datasource.url=jdbc:postgresql://<YOUR_DB_HOST>:5432/athena_db
3 spring.datasource.username=admin_user
4 spring.datasource.password=\${DB_PASSWORD_ENV_VAR}
5 spring.jpa.hibernate.ddl-auto=update
```

4.2 2. Enabling JPA Repositories

In `ResourceController.java`, replace the static mock list with the JPA service call:

```
1 // REPLACE THIS:
2 // return Arrays.asList(new Resource(...));
3
4 // WITH THIS:
5 @Autowired
6 private ResourceRepository repository;
7
8 @GetMapping
9 public List<Resource> getAllResources() {
10     return repository.findAll();
11 }
```

5 Deployment Pipeline

The system is fully automated via **Azure DevOps**.

- **Triggers:** Pushes to main branch.
- **Quality Gate:** Maven runs `ResourceControllerTest.java`. If tests fail, the build is rejected.
- **Artifacts:** Generates a Docker-ready JAR file and optimized Angular static assets.